

R

RetailIQ

AI E-commerce Intelligence

FULL PROJECT PRESENTATION

RetailIQ

AI Powered E-commerce Intelligence Platform

React SaaS dashboard + FastAPI backend + machine learning churn prediction + customer segmentation.

Records processed

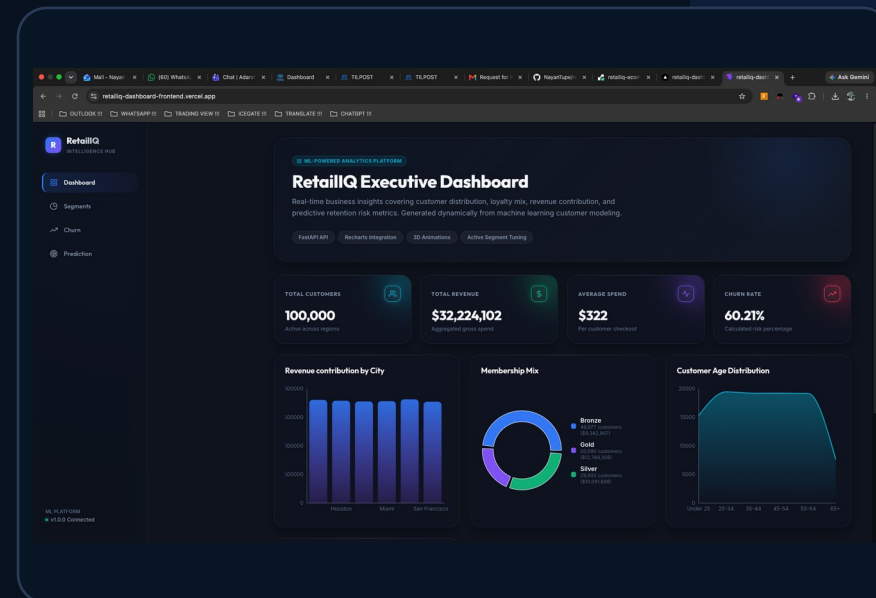
100,000

Churn recall

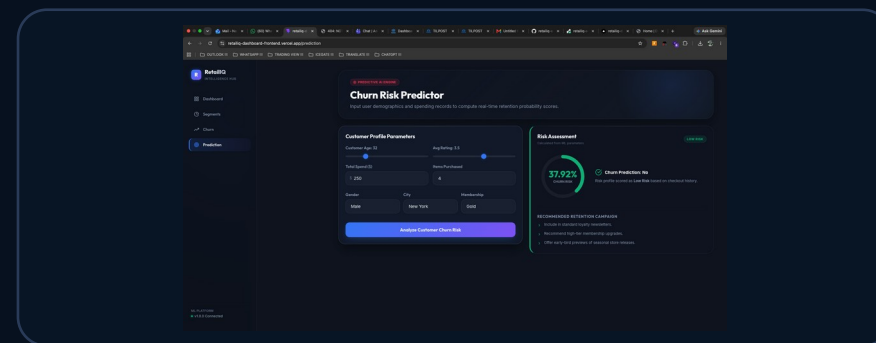
96.09%

Deployment

Render + Vercel



Production dashboard screenshot



Latest churn predictor screenshot

Executive Snapshot

What this project proves in one view.

Business problem

E-commerce owners need early warning for customer churn, revenue leakage, and low-value segments.

AI solution

RetailIQ predicts churn risk, groups customers, and converts analytics into retention actions.

Production proof

Frontend deployed on Vercel, backend on Render, and APIs tested with Swagger.

Portfolio value

Demonstrates full-stack React, FastAPI, ML pipeline, API integration, dashboards, and deployment.



Business Problem and Solution

RetailIQ turns raw customer data into owner-ready actions.

Challenges for e-commerce teams

Customers silently stop buying

Owners do not know which segment needs attention

Retention campaigns are usually generic

Revenue risk is hidden across city, membership, and rating behavior



RetailIQ solution

Predicts churn probability in real time

Segments customers with K-Means clustering

Shows dashboards for revenue, membership, age, and churn

Recommends focused retention campaigns

Technology Stack

End-to-end AI SaaS architecture across frontend, backend, ML, and deployment.

Frontend

React.js (Vite)
Axios
Framer Motion
Recharts / Plotly
Glassmorphism UI

Backend

FastAPI
Uvicorn
Pydantic
CORS middleware
REST APIs

Machine Learning

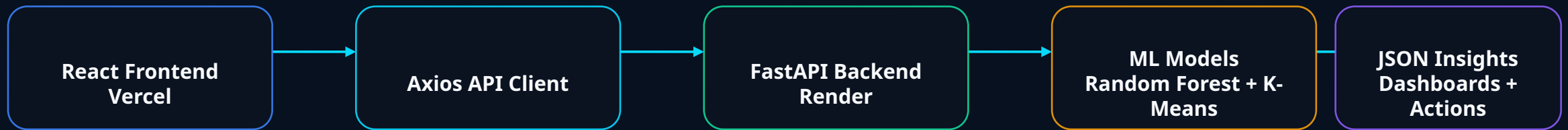
Scikit-learn
Random Forest
K-Means clustering
Pandas
Joblib models

Deployment

Vercel frontend
Render backend
GitHub repos
Docker-ready backend
Swagger docs

System Architecture

A production-style flow from React UI to ML-powered decisions.



Frontend pages consume analytics endpoints and the prediction endpoint. Backend loads trained model files, validates request data, engineers features, and returns business-friendly JSON output.

Machine Learning Pipeline and Performance

Leakage-safe model design with business-focused recall.



Why recall matters: for churn prediction, missing a risky customer can cause revenue loss. High recall helps owners target customers before they leave.

Backend Code Architecture

Actual backend ZIP was used to summarize the FastAPI structure.



Backend folder map

```
retailiq-ecommerce-ml-backend/  
api/  
  main.py  
  routes/  
    prediction_routes.py  
    dashboard_routes.py  
    churn_routes.py  
  schemas/  
    prediction_schema.py  
  services/  
    prediction_service.py  
models/  
  churn_model.pkl  
  segmentation_model.pkl  
reports/  
  api_documentation.md  
  model_performance_report.md
```

REST API surface

```
GET /  
POST /predict/churn  
GET /analytics/summary  
GET /analytics/segments  
GET /analytics/churn-by-membership  
GET /analytics/details
```

Backend Code Proof: FastAPI App and Routes

Selected snippets from the uploaded backend ZIP.

api/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from api.routes.prediction_routes import router as prediction_router
from api.routes.dashboard_routes import router as dashboard_router

app = FastAPI(title="RetailIQ API", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "https://retailiq-dashboard-frontend.vercel.app"
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(prediction_router)
app.include_router(dashboard_router)
```

prediction_routes.py

```
router = APIRouter(prefix="/predict", tags=["Prediction"])

class CustomerInput(BaseModel):
    age: int
    total_spend: float
    items_purchased: int
    average_rating: float

@router.post("/churn")
def predict_churn(data: CustomerInput):
    df = pd.DataFrame([data.dict()])
    prediction = model.predict(df)[0]
    probability = model.predict_proba(df)[0][1]
    return {
        "churn_prediction": "Yes" if prediction == 1 else "No",
        "churn_probability": round(probability * 100, 2),
        "risk_level": "High Risk" if probability > 0.7 else "Low Risk"
    }
```


Backend Prediction Service

Model loading, risk scoring, and recommendation generation.

api/services/prediction_service.py

```
MODEL_PATH = Path("models/churn_model.pkl")

def load_churn_model():
    if not MODEL_PATH.exists():
        raise FileNotFoundError(
            f"Model not found at {MODEL_PATH}."
        )
    return joblib.load(MODEL_PATH)

def get_risk_level(probability: float) -> str:
    if probability >= 0.75:
        return "High Risk"
    elif probability >= 0.50:
        return "Medium Risk"
    return "Low Risk"

def get_recommendation(risk_level: str) -> str:
    recommendations = {
        "High Risk": "Offer retention discount, loyalty reward, or personal follow-up.",
        "Medium Risk": "Send personalized offers and monitor customer engagement.",
        "Low Risk": "Maintain regular communication and loyalty benefits."
    }
    return recommendations.get(risk_level, "No recommendation available.")
```

Why this matters

Model file is loaded from backend models folder

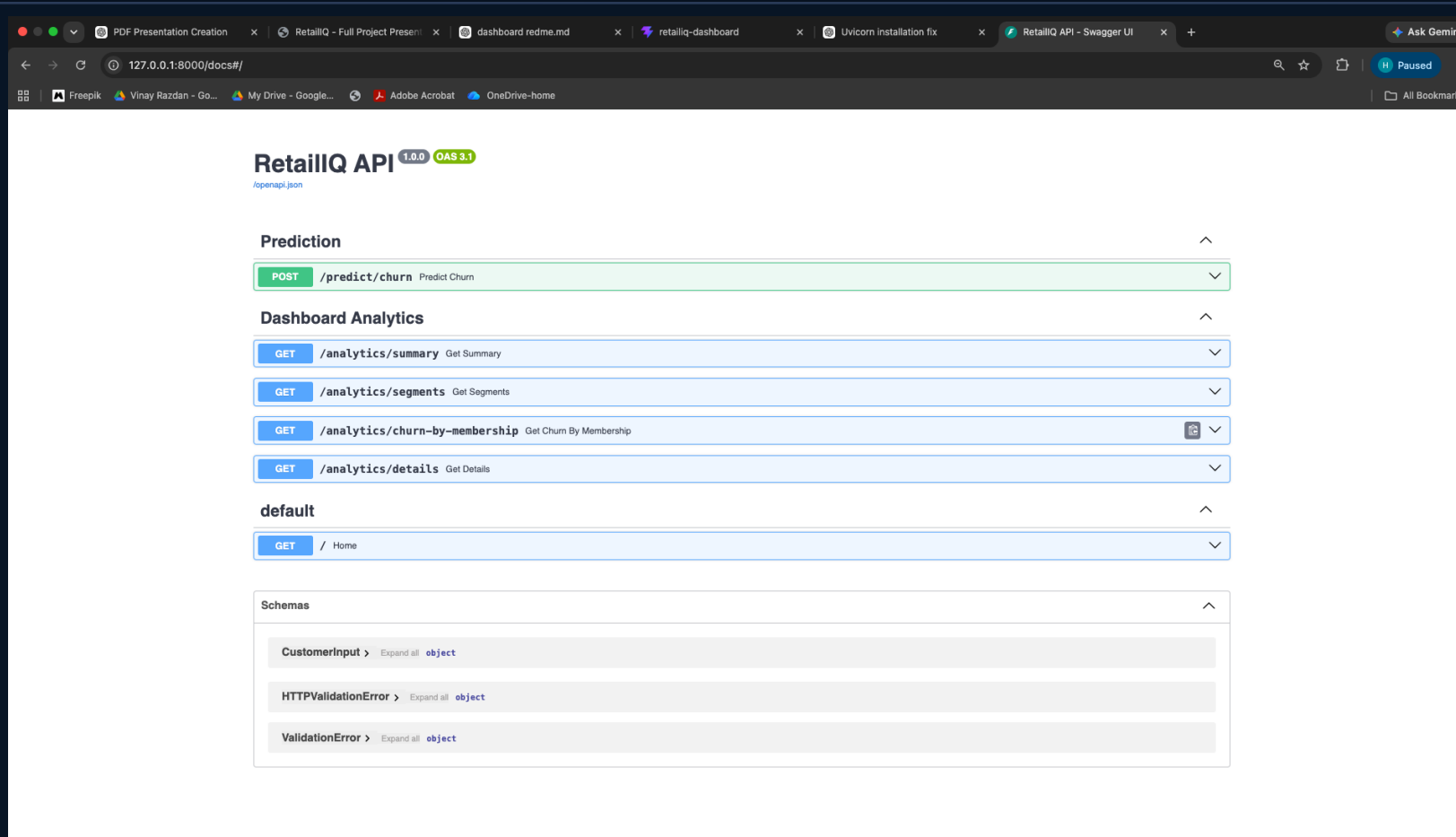
Risk thresholds convert probability into business-friendly labels

Recommendation text helps owners take action quickly

Service layer keeps API route clean and scalable

API Documentation and Testing

Swagger UI shows available backend endpoints for prediction and analytics.



Swagger documentation with Prediction and Dashboard Analytics endpoints

Live API Request Testing

POST /predict/churn request body validated through Swagger.

The screenshot shows the Swagger UI interface for a REST API. The browser's address bar displays the URL `127.0.0.1:8000/docs#/Prediction/predict_churn_predict_churn_post`. The interface is titled "Prediction" and shows a "POST /predict/churn Predict Churn" endpoint. The "Parameters" section indicates "No parameters". The "Request body" section is marked as "required" and has a dropdown menu set to "application/json". Below this, the "Edit Value | Schema" section contains a JSON object:

```
{  "age": 32,  "total_spend": 250,  "items_purchased": 4,  "average_rating": 3.5,  "gender": "Male",  "city": "New York",  "membership": "Gold"}
```

. At the bottom of the request section is a blue "Execute" button and a "Clear" button. The "Responses" section is currently empty. Below the "Responses" section, the "Curl" section shows the equivalent curl command:

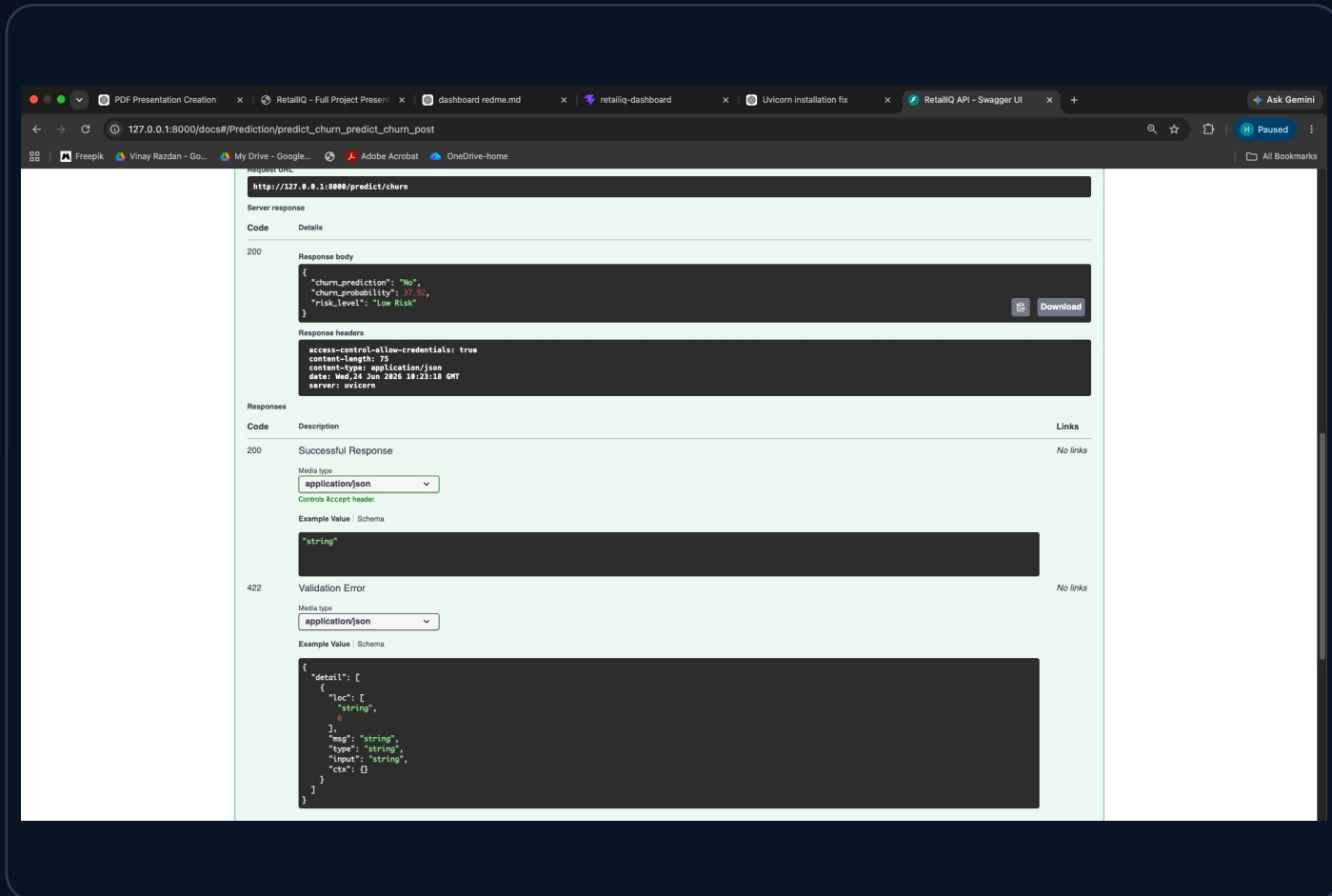
```
curl -X 'POST' \  'http://127.0.0.1:8000/predict/churn' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "age": 32,  "total_spend": 250,  "items_purchased": 4,  "average_rating": 3.5,  "gender": "Male",  "city": "New York",  "membership": "Gold"  }'
```

. The "Request URL" section at the bottom shows `http://127.0.0.1:8000/predict/churn`.

Request body with customer age, spend, purchase count, rating, gender, city, and membership

Backend API Response Proof

Successful response from POST /predict/churn.



Swagger Server response: 200 with JSON response body

Response body

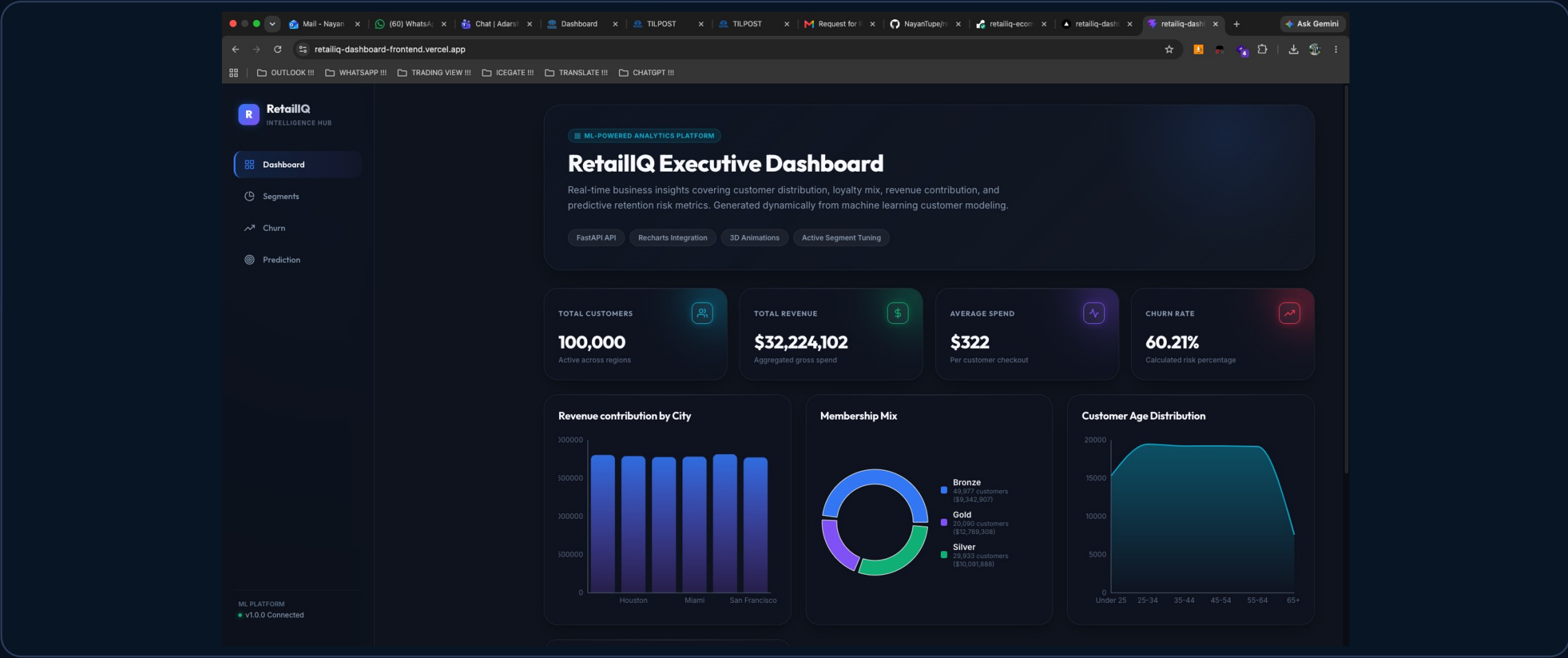
```
{
  "churn_prediction": "No",
  "churn_probability": 37.92,
  "risk_level": "Low Risk"
}
```

Proof points

- Endpoint returns HTTP 200
- Model score matches frontend result
- Risk label is business-readable
- Ready for React API integration

Frontend Dashboard

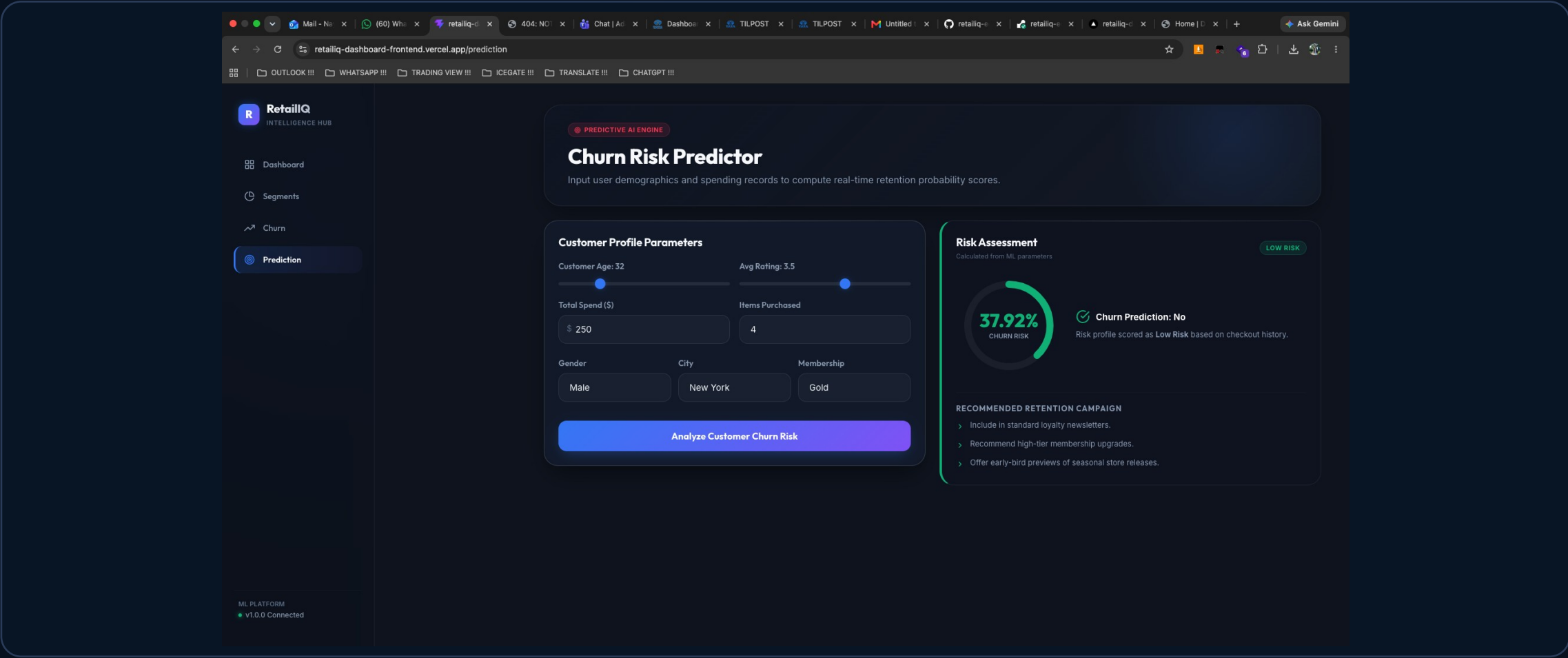
Executive dashboard for customer, revenue, membership, age, and churn KPIs.



RetailIQ Executive Dashboard - production UI screenshot

Churn Risk Predictor

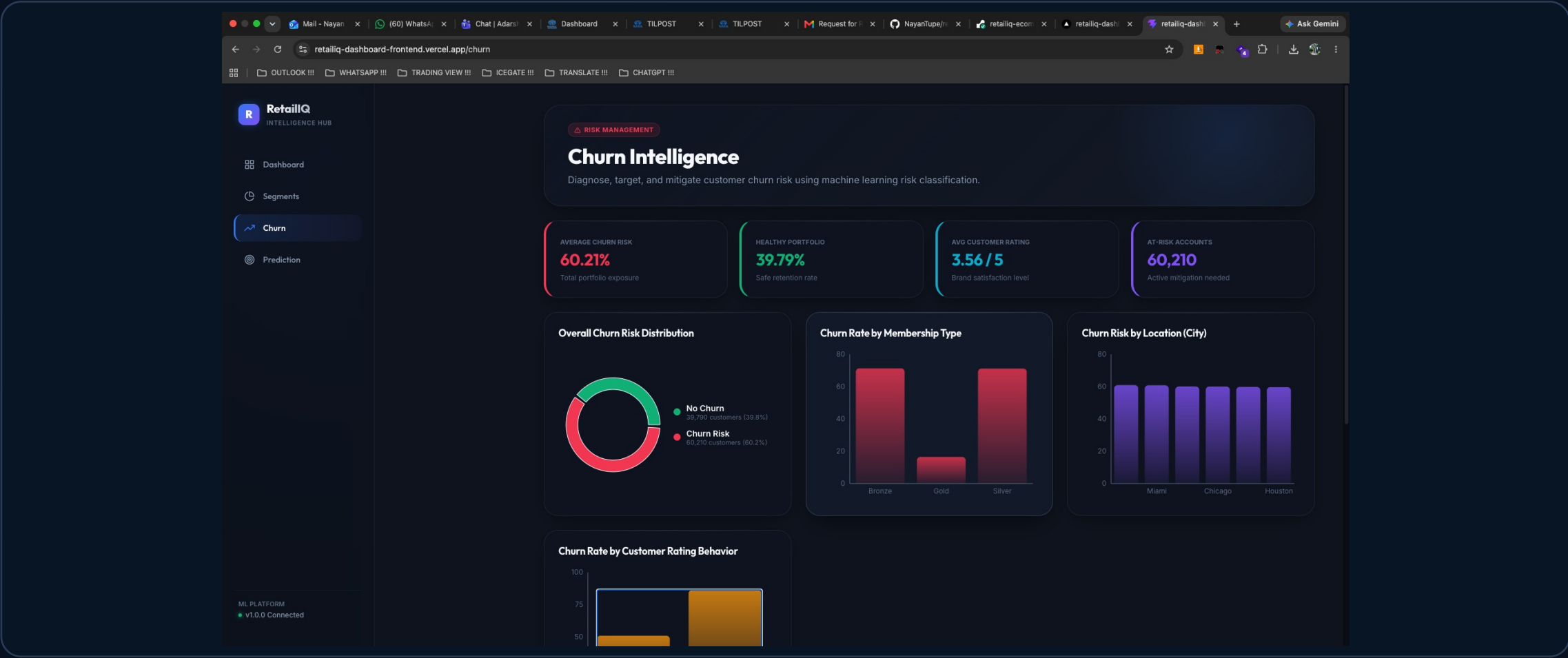
Latest screenshot replaced exactly as requested - full-fit and not clipped.



Prediction page showing 37.92% churn risk and Low Risk output

Churn Intelligence

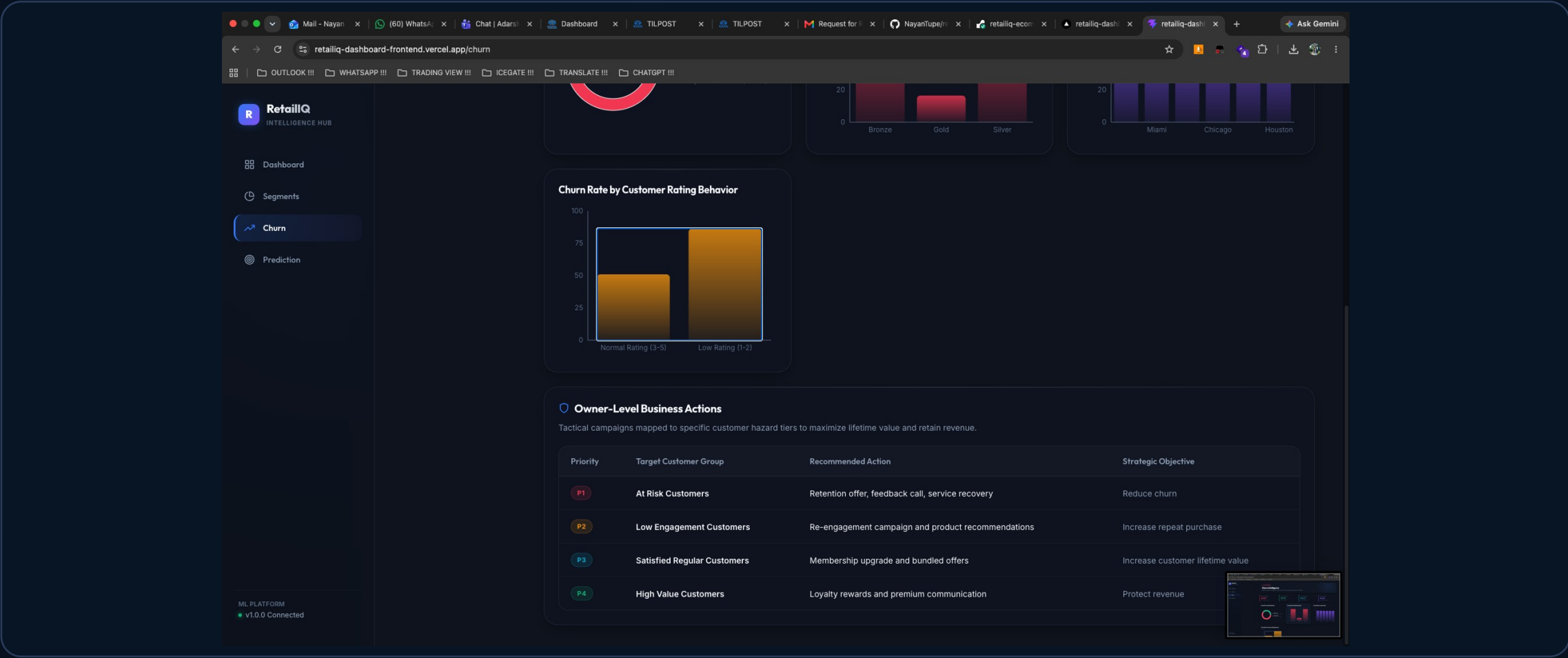
Risk management analytics for portfolio exposure and at-risk accounts.



Churn Intelligence view with KPI cards and distribution charts

Owner-Level Business Actions

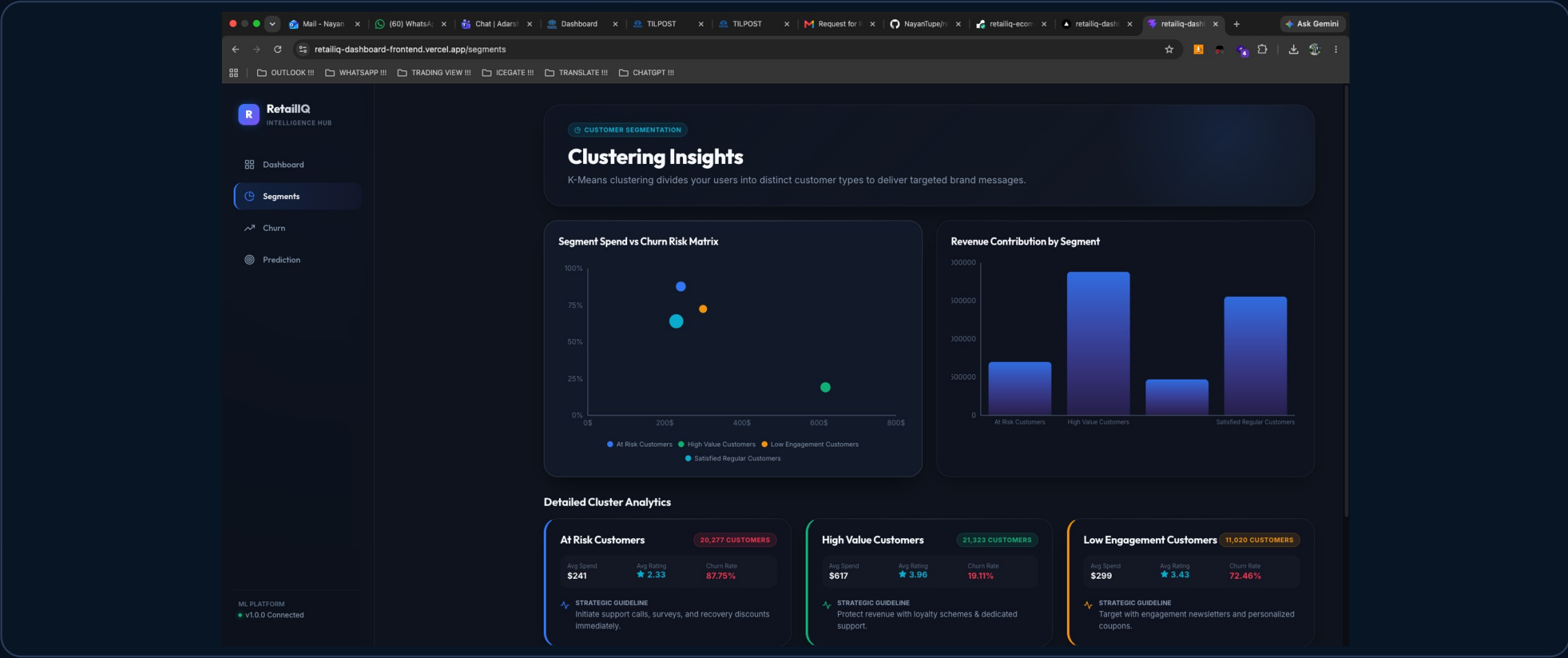
Converts churn analytics into retention actions by priority group.



Recommended actions table for owner decision-making

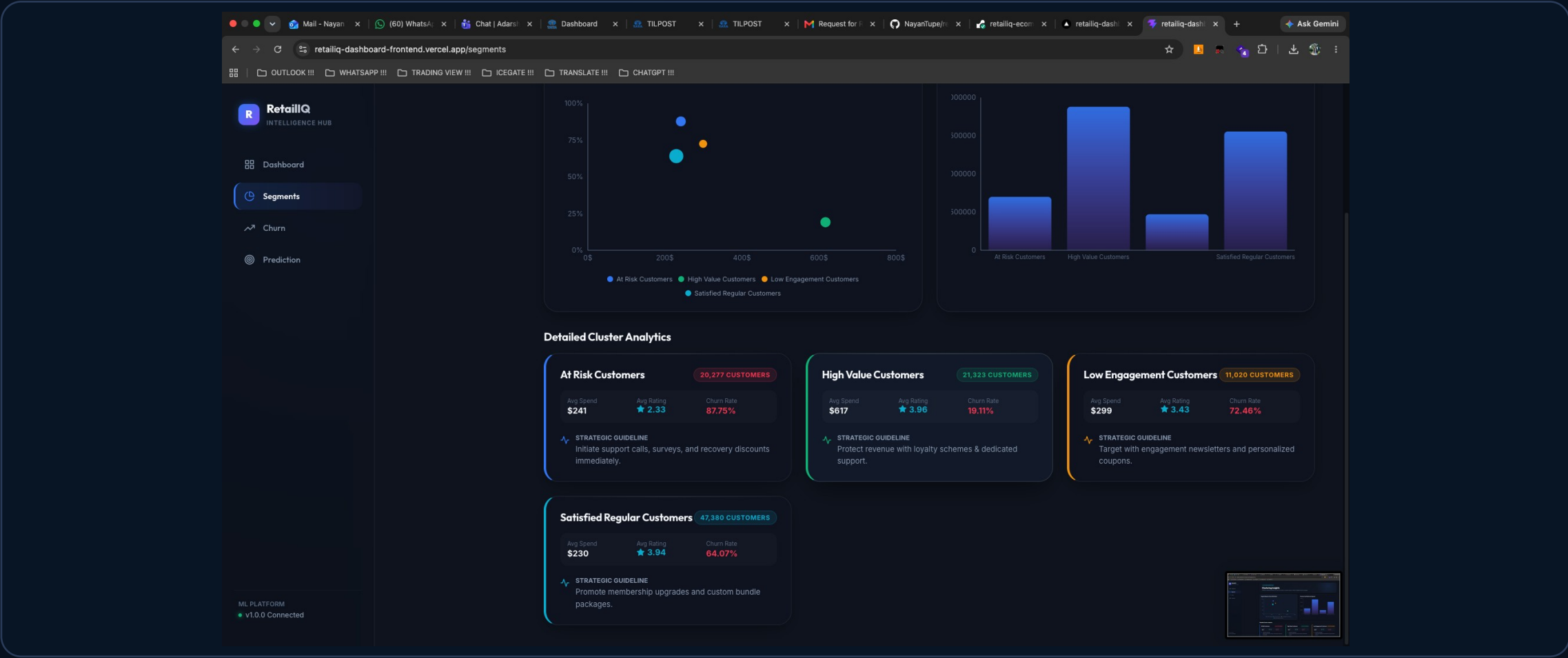
Customer Segmentation Insights

K-Means clustering groups users into actionable customer segments.



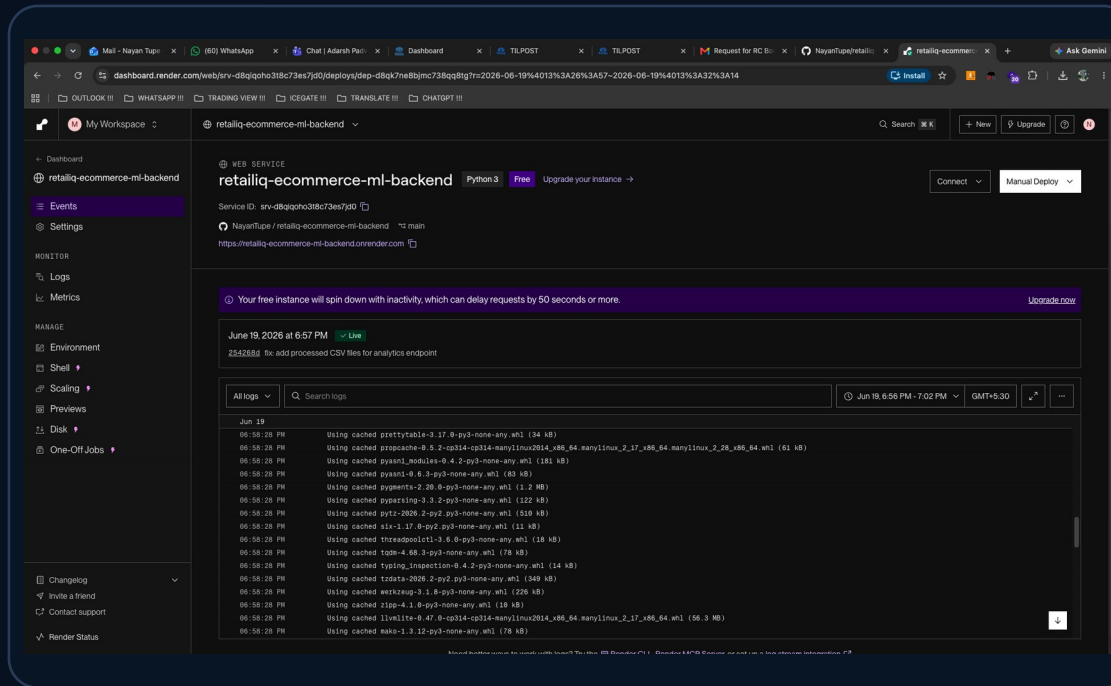
Detailed Cluster Analytics

Segment cards show customer count, spend, rating, churn rate, and strategic guideline.

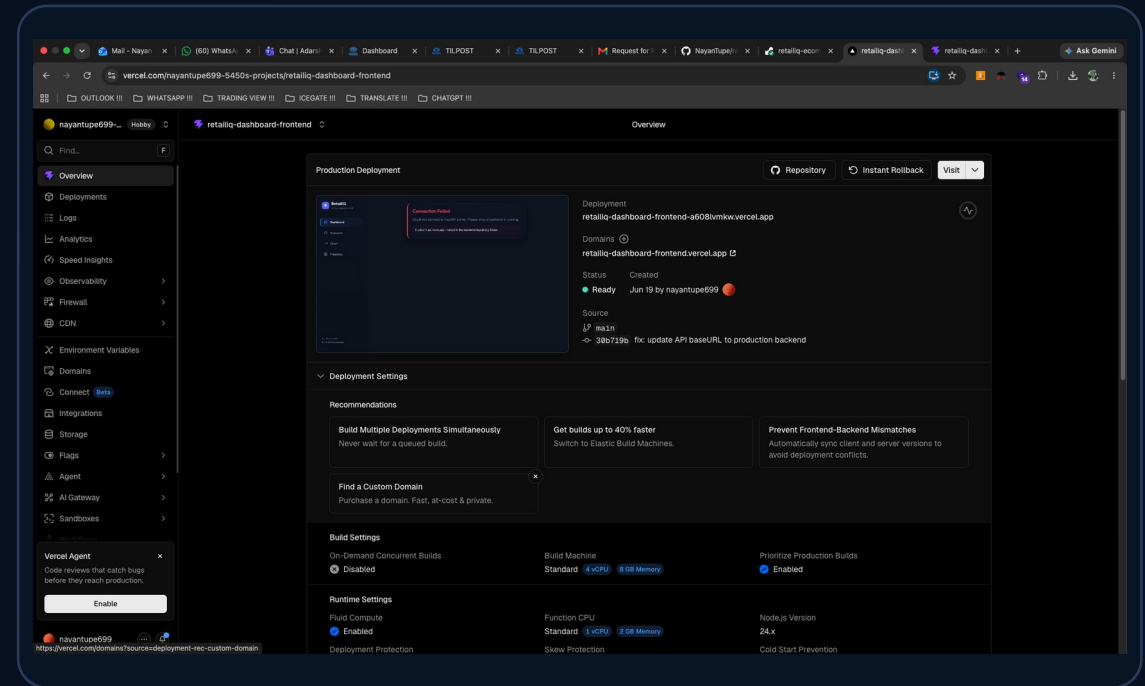


Deployment Evidence

Frontend and backend are deployment-ready with public hosting workflows.



Render backend deployment evidence

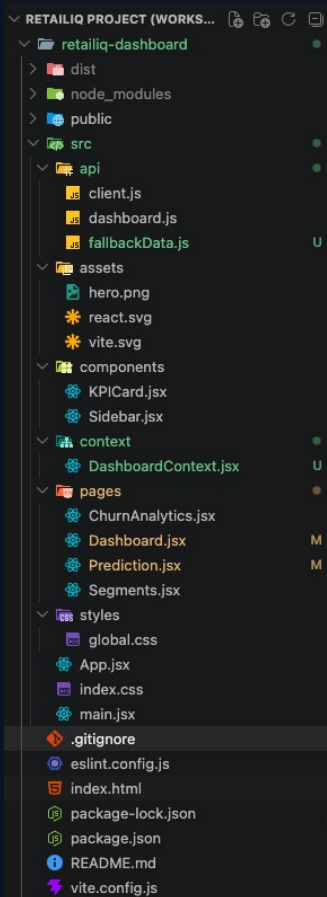


Vercel frontend production deployment evidence

Frontend: <https://retailiq-dash-board-frontend.vercel.app/> | Backend APIs: Render FastAPI service | Repos: GitHub version control

Frontend Repository Structure

React Vite architecture with API layer, reusable components, and dashboard pages.



Frontend source structure screenshot

Clean React organization

```
src/  
  api/  
    client.js  
    dashboard.js  
    fallbackData.js  
  components/  
    KPICard.jsx  
    Sidebar.jsx  
  context/  
    DashboardContext.jsx  
  pages/  
    Dashboard.jsx  
    Prediction.jsx  
    ChurnAnalytics.jsx  
    Segments.jsx  
  styles/  
    global.css  
  App.jsx  
  index.css  
  main.jsx
```

Frontend value

Dedicated API client for backend calls

Separate pages for major business workflows

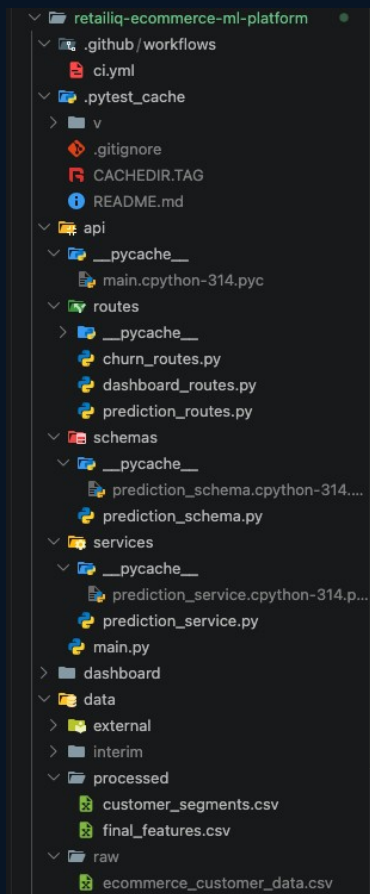
Reusable KPI and navigation components

Fallback data support improves UI stability

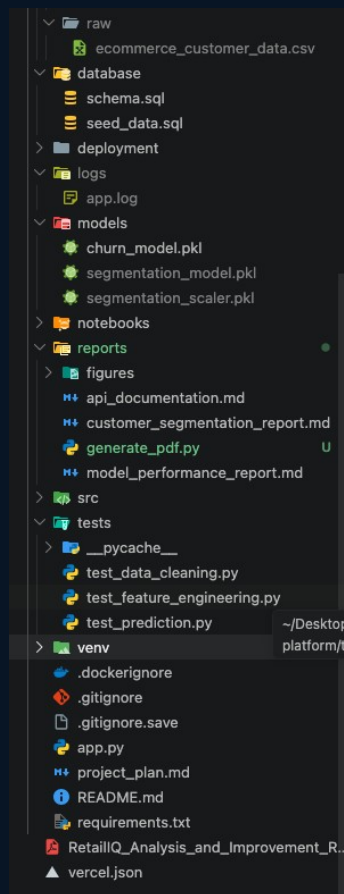
Responsive SaaS-style presentation

Backend Repository Structure

API, data, models, notebooks, reports, tests, and deployment files in one backend repo.



Backend API/routes/services/models structure



Data, database, reports, tests, deployment structure

Backend value

Training notebooks preserved for explainability

Processed CSVs support analytics endpoints

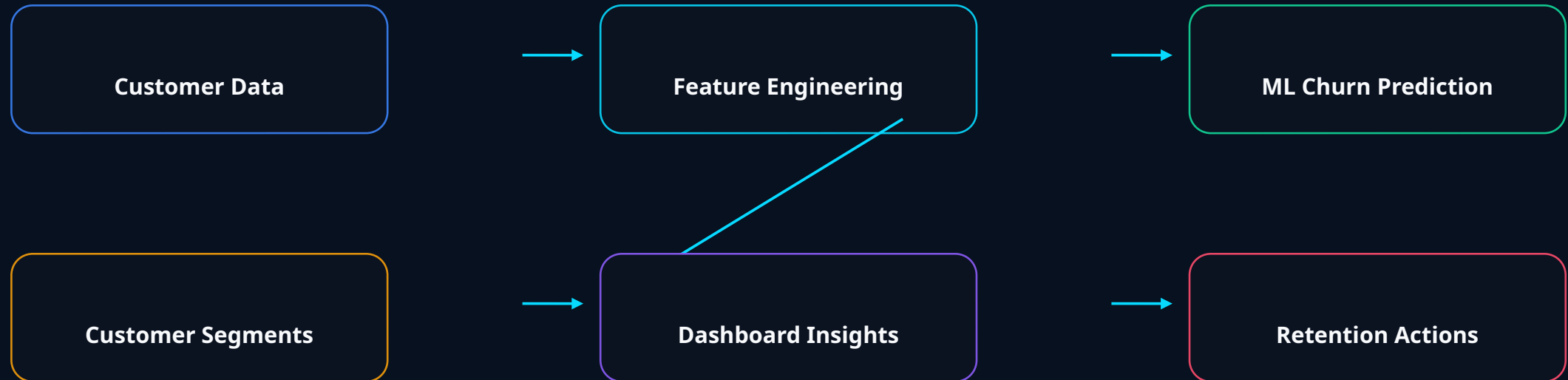
Reports document ML performance and segmentation

Tests show engineering discipline

Docker and CI files support production workflow

From Data to Business Decision

How RetailIQ helps owners act instead of just watching charts.



Owner decisions supported: retention offers, support follow-ups, loyalty rewards, membership upgrades, and segment-wise campaigns.

Client and Owner Benefits

Business value explained in owner-friendly language.

Reduce churn

Identify risky customers before revenue is lost.

Increase lifetime value

Protect high-value customers with loyalty actions.

Better targeting

Use segments to personalize campaigns and offers.

Faster decisions

KPIs, charts, and recommended actions in one dashboard.

Operational clarity

See churn by city, membership, rating, and segment.

Scalable foundation

Ready for auth, role-based access, and real-time analytics.

Recruiter and Job Portfolio Value

This project demonstrates practical full-stack + AI engineering ability.

Frontend skills

- React Vite application
- Reusable components
- API integration
- Interactive charts
- Responsive UI

Backend skills

- FastAPI REST APIs
- Pydantic validation
- CORS setup
- ML model loading
- Swagger documentation

ML + deployment

- Random Forest model
- K-Means segmentation
- Leakage-safe reporting
- Render + Vercel deployment
- GitHub repositories

Roadmap and Closing

Clear next steps to move from portfolio SaaS to production SaaS.

Future improvements

- JWT authentication and login/signup
- Role-based dashboards for owner, analyst, and admin
- Real-time WebSocket analytics updates
- Scheduled model retraining pipeline
- AI chatbot assistant for business questions
- CI/CD and monitoring enhancements

Final positioning

RetailIQ is a complete AI SaaS portfolio project: a React analytics dashboard connected to a FastAPI backend that serves ML-powered churn predictions, segmentation insights, business recommendations, and production deployment proof.

Useful for: GitHub README, client presentations, owner demos, interviews, internships, and full-stack/ML job applications.